Argument 5: For small models, we can explain mechanistically how induction heads work, and can show they contribute to in-context learning. Furthermore, the actual mechanism of operation suggests natural ways in which it could be re-purposed to perform more general in-context learning.

STRENGTH OF ARGUMENT FOR SUB-CLAIMS

	Small Attention-Only	Small with MLPs	Large Models
Contributes Some	Strong, Mechanistic	Strong, Mechanistic	Medium, Mechanistic
Contributes Majority	Weak, Mechanistic		

One of the main reasons we care about whether induction heads drive in-context learning is that we can understand them and so have a path to understanding in-context learning. But we can also turn this around: we can use our understanding of induction heads to make a purely logical argument that they *should* contribute to in-context learning.

We begin with a semi-empirical argument. Let's take for granted that induction heads behave the way we've described and empirically seen, searching the context for previous examples and copying what happened next. We should expect such a procedure to improve a model's ability to predict tokens later in its context. We're essentially using the previous context as data points for a nearest neighbor algorithm, and nearest neighbors improves as one gives it more data points. Therefore, if induction heads exist as described, they would contribute to in-context learning as we've defined it.

In some sense, this argument is quite strong if we're only arguing that there exist some cases where induction heads contribute to in-context learning. We've seen concrete examples above where induction heads improve token predictions by copying earlier examples. If nothing else, they must help in those cases! And more generally, our definition of induction heads (in terms of their behavior on repeated random sequences) suggests they behave this way quite generally. This argument doesn't say anything about *what fraction* of in-context learning is performed by induction heads, but it seems like a very strong argument that *some* is, both in large and small models.

But the really satisfying thing about this line of attack — namely, using our understanding of induction heads to anticipate their impact on in-context learning — is that we can actually drop the dependency on empirical observations of induction head behavior, at the cost of needing to make a more complex argument. In our previous paper, we were able to reverse engineer induction heads, showing from the parameter level how they implement induction behavior (and that they should). If we trust this analysis, we can know how induction heads behave without actually running them, and the argument we made in the previous paragraph goes through. Of course, there are some limitations. In the previous paper, we only reverse engineered a single induction head in a small attention-only model, although we can reverse engineered others (and have done so). A bigger issue is that right now we're unable to reverse engineer induction heads in models with MLP layers. But at least in some cases we've observed, we can look at the parameters of a transformer and identify induction heads, just as a programmer might identify an algorithm by reading through source code.

We can actually push this argument a bit further in the case of the two-layer attention only transformer we reverse engineered in the previous paper. Not only do we understand the induction heads and know that they should contribute to in-context learning, but there doesn't seem to really be an alternative mechanism that could be driving it.[23] This suggests that induction heads are the primary driver of in-context learning, at least in very small models.

The following section will briefly summarize reverse engineering induction heads. Note that it relies heavily on linking to our previous paper. *We do not expect it to be possible to follow without reading the linked portions.* After that, we briefly discuss how the described mechanism could also implement more abstract types of induction head behavior.

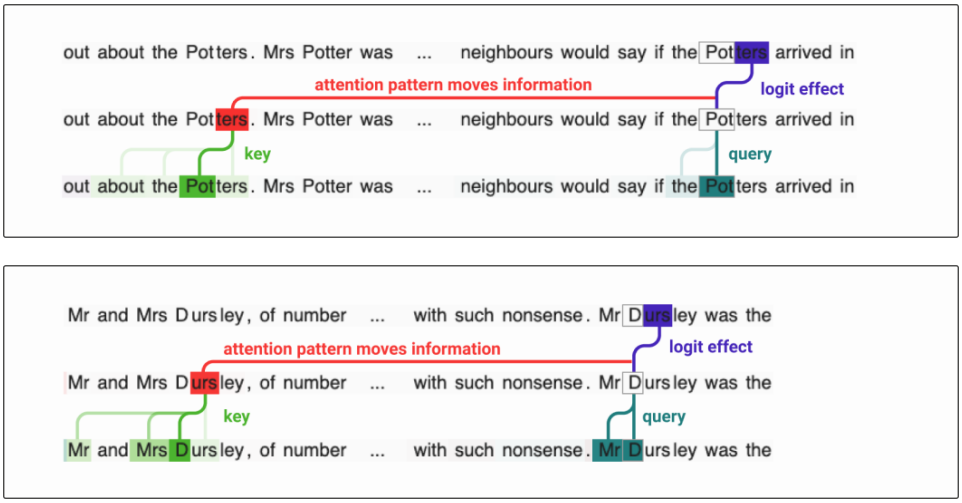
Summary of Reverse Engineering Induction Heads

Note: This section provides a dense summary with pointers to our previous paper; please see the previous paper for more information.

Recall from Key Concepts that induction heads are defined as heads that exhibit both *copying* and *prefix matching*.

***Copying* is done by the OV ("Output-Value") circuit.** One of the defining properties of an induction head is that it copies. Induction heads are not alone in this! Transformers seem to have quite a number of copying heads, of which induction heads are a subset. This is done by having a "copying matrix" OV circuit, most easily characterized by its positive eigenvalues.

***Prefix matching* is implemented with K-composition (and to a lesser extent Q-composition) in the QK ("Query-Key") Circuit.** In order to do prefix matching, the key vector at the attended token needs to contain information about the *preceding* tokens — in fact, information about the *attended token itself* is quite irrelevant to calculating the attention pattern for induction.[24] In the models we study, "key shifting" occurs primarily using what we call K-composition. That is to say that an induction head's W_K reads from a subspace written to by an earlier attention head. The most basic form of an induction head uses pure K-composition with an earlier "previous token head" to create a QK-Circuit term of the form $\mathrm{Id} \otimes h_{prev} \otimes W$ where W has positive eigenvalues. This term causes the induction head to compare the *current* token with every earlier position's *preceding* token and look for places where they're similar. More complex QK circuit terms can be used to create induction heads which match on more than just the preceding token.



Induction heads use previous heads to shift key information and match it against the present token. As they get more sophisticated, they also shift query information.

Combined, these are a detectable mechanism for induction heads. In the small models we studied in our previous paper, all induction heads have the described QK term driving their attention pattern and a positive eigenvalue circuit performing copying.

Some models use a different mechanism to implement induction heads. In GPT-2 [10], we've seen evidence of a second "pointer-arithmetic" mechanism for induction heads. This mechanism makes use of the positional embedding and "Q-composition". In GPT-2, the earlier attention head attends to previous copies of the current token, and its $W_O V$ circuit copies their positional embedding into a subspace in the present token. The induction head then uses Q-composition to rotate that position embedding one token forward, and thereby attend to the following token. This mechanism isn't available to the models we study here, since they do not add positional information into the residual stream.[25]

What About More Complex Induction Heads?

What about the induction heads we saw in Argument 2 with more complex behavior? Can we also reverse engineer them? Do they operate on the same mechanisms? Presently, fully reverse engineering them is beyond us, since they exist in large models with MLPs, which we don't have a strong framework for mechanistically understanding. However, we hypothesize they're different in two ways: (1) using more complex QK terms rather than matching on just the previous token; and (2) matching and copying more abstract and sophisticated linguistic features, rather than precise tokens.

When we first introduced induction heads, we observed that they could be seen as a kind of "in-context nearest neighbor" algorithm. From this perspective, it seems natural that applying the same mechanism to more abstract features can produce more complex behavior.